

Chapter 3

Pattern Printing with Loops

Solid Patterns

1. Right Triangle Star Pattern

- Outer loop i: rows 1 to 5. Inner loop j: 1 to i. Stars per row increase with the row number.

```
for (int i = 1; i <= 5; i++) {  
    for (int j = 1; j <= i; j++) {  
        System.out.print("* ");  
    }  
    System.out.println();  
}
```

```
*  
* *  
* * *  
* * * *  
* * * * *
```

2. Inverted Right Triangle Star Pattern

- Outer loop i runs backward: 5 down to 1 (i--). Inner loop j: 1 to i, same as before, but shrinks as i shrinks.

```
for (int i = 5; i >= 1; i--) {  
    for (int j = 1; j <= i; j++) {  
        System.out.print("* ");  
    }  
    System.out.println();  
}
```

```
* * * * *  
* * * *  
* * *  
* *  
*
```

3. Right-Aligned Triangle Star Pattern

- Two inner loops per row: the first prints leading spaces (decreasing count), the second prints the stars (increasing count) — together they push the triangle to the right.

```
for (int i = 1; i <= 5; i++) {  
    for (int j = 5; j > i; j--) {
```

```

        System.out.print(" "); // leading spaces
    }
    for (int k = 1; k <= i; k++) {
        System.out.print("* "); // stars
    }
    System.out.println();
}

```

```

    *
   * *
  * * *
 * * * *
* * * * *

```

4. Pyramid Star Pattern

- Same two-part idea as the right-aligned triangle, but the star count uses the odd-number formula ($2*i - 1$) so the pyramid is symmetric, and leading spaces use (rows - i) to center it.

```

int rows = 5;
for (int i = 1; i <= rows; i++) {
    for (int j = rows; j > i; j--) {
        System.out.print(" "); // leading spaces
    }
    for (int k = 1; k <= (2 * i - 1); k++) {
        System.out.print("*"); // stars (odd count per row)
    }
    System.out.println();
}

```

```

    *
   ***
  *****
 * * * * *
* * * * *

```

Hollow Patterns

Hollow patterns print symbols only on the outer border of a shape and leave the inside empty. The key technique is a single if-else condition inside the inner loop: print a symbol on border cells, and a space everywhere else.

1. Hollow Square Star Pattern

- A cell (i, j) is on the border if it's in the first/last row ($i == 1$ or $i == n$) OR the first/last column ($j == 1$ or $j == n$).

- If it's a border cell, print a star; otherwise print a space.

```
int n = 5;
for (int i = 1; i <= n; i++) {
    for (int j = 1; j <= n; j++) {
        if (i == 1 || i == n || j == 1 || j == n)
            System.out.print("* ");
        else
            System.out.print("  ");
    }
    System.out.println();
}
* * * * *
*       *
*       *
*       *
*       *
* * * * *
```

2. Hollow Rectangle Star Pattern

- Exactly the same border logic as the hollow square, but rows and columns are set independently (rows = 4, cols = 7), so the shape isn't square.

```
int rows = 4, cols = 7;
for (int i = 1; i <= rows; i++) {
    for (int j = 1; j <= cols; j++) {
        if (i == 1 || i == rows || j == 1 || j == cols)
            System.out.print("* ");
        else
            System.out.print("  ");
    }
    System.out.println();
}
* * * * * * *
*       *
*       *
* * * * * * *
```

3. Hollow Right Triangle Star Pattern

- Same hollow idea, but applied inside a triangle instead of a rectangle. The inner loop only runs up to i (not up to a fixed n), so the shape is triangular.
- A cell is on the border if it's the first column ($j == 1$), the last column of that row ($j == i$), or the last row overall ($i == n$) — that traces the two slanted/flat edges plus the base.

```
int n = 5; // height of triangle
```

```

for (int i = 1; i <= n; i++) {
    for (int j = 1; j <= i; j++) {
        if (j == 1 || j == i || i == n)
            System.out.print("* ");
        else
            System.out.print("  ");
    }
    System.out.println();
}

```

```

*
* *
*  *
*    *
* * * * *

```

Tips for Solving Pattern Printing Problems

Use this checklist for any new pattern question, solid or hollow:

1. Count the rows first. That becomes your outer loop's range, e.g. for (int i = 1; i <= n; i++).
2. Find the relationship between row number and symbol count. If row 1 has 1 star and row n has n stars, the inner loop runs 1 to i. If it's the reverse, use (n - i + 1).
3. Decide the outer loop's direction. Counting up (1 -> n) builds an upright shape; counting down (n -> 1) inverts it — just switch i++ to i-- and flip the starting value.
4. For aligned/pyramid shapes, split the row into two inner loops. One loop prints the leading spaces (usually shrinking as i grows), the second prints the actual symbols. Order matters: spaces first, then symbols.
5. For pyramids, remember the odd-number rule: printing (2*i - 1) symbols per row (instead of i) is what makes each row grow symmetrically from a center point.
6. For hollow shapes, think in terms of "border vs. interior", not row-by-row counting. Use a single if condition that checks all four edges — first row, last row, first column, last column — and print a space for anything that fails all of them.
7. For a hollow triangle, the border condition changes slightly: only 3 edges exist (left slanted edge j == 1, right slanted edge j == i, and the flat base i == n) since a triangle doesn't have a separate top edge.
8. Keep the inner loop(s) focused on one row only. Never let inner-loop logic depend on what happened in a previous row — that's the outer loop's job.
9. Always call System.out.println() after the inner loop(s), never inside them — that ends the row exactly once, after all columns are printed.
10. Use print(), not println(), inside inner loops, since a row is built piece by piece before moving to a new line.
11. Trace 2–3 rows by hand before coding. Write out the values of i and j for the first few rows on paper — this reveals the exact loop bounds and conditions needed.

12. To turn a star pattern into a number pattern, replace the fixed symbol (e.g. "*") with a variable or expression like i, j, or i * j.

Quick Reference Table

Pattern Type	Outer Loop	Inner Loop Logic
Upright triangle	i = 1 to n	print i stars (j = 1 to i)
Inverted triangle	i = n to 1	print i stars (j = 1 to i)
Right-aligned triangle	i = 1 to n	spaces: (n - i); then i stars
Pyramid	i = 1 to n	spaces: (n - i); then (2i - 1) stars
Hollow square/rectangle	i = 1 to rows	star if edge row/col, else space
Hollow triangle	i = 1 to n	star if j==1 or j==i or i==n, else space